Mandate

Spending guardrails for AI shopping agents, settled with PayPal.

> The model proposes. The code disposes.

54

tests, policy through PayPal

$115.00

refused above the threshold, then settled in sandbox

0

credentials needed to run the demo

An agent that can buy needs authority. Today that authority is a card.

Give the agent a card and it can spend anywhere the card clears. The controls live at the bank: a monthly cap, maybe a merchant lock on a virtual card. Coarse, set elsewhere, invisible to the agent, and silent about *this* cart.

Withhold the card and the agent researches, compares and fills a basket — then hands the last step back to you, because you are not giving software your credential.

> You can cap the card at your bank. You cannot tell the agent what it may buy.

CONTROL	WHERE IT LIVES
Monthly card limit	`Issuer`
Merchant lock	`Virtual card`
Per-cart rule with a reason	`Mandate`

A person already writes these limits down — in a note on their phone, which the merchant ignores. Mandate makes them executable.

For people who delegate a bounded purchase, not for people who want to fill in forms.

The audience: someone buying a specific, bounded thing — the week's groceries, school supplies, a replacement part — who will let an agent transact but will not hand over a blank card.

What they actually do: state the goal in one sentence. The app drafts the limits from the catalogue. They look at one line and approve it, or change one number. Seven fields exist, but nobody has to open them.

01

Say the goal

"Trail shoes for the weekend, under the cap."

02

Approve the limits

Budget, cap, threshold, merchants — drafted, one tap.

03

Let it buy

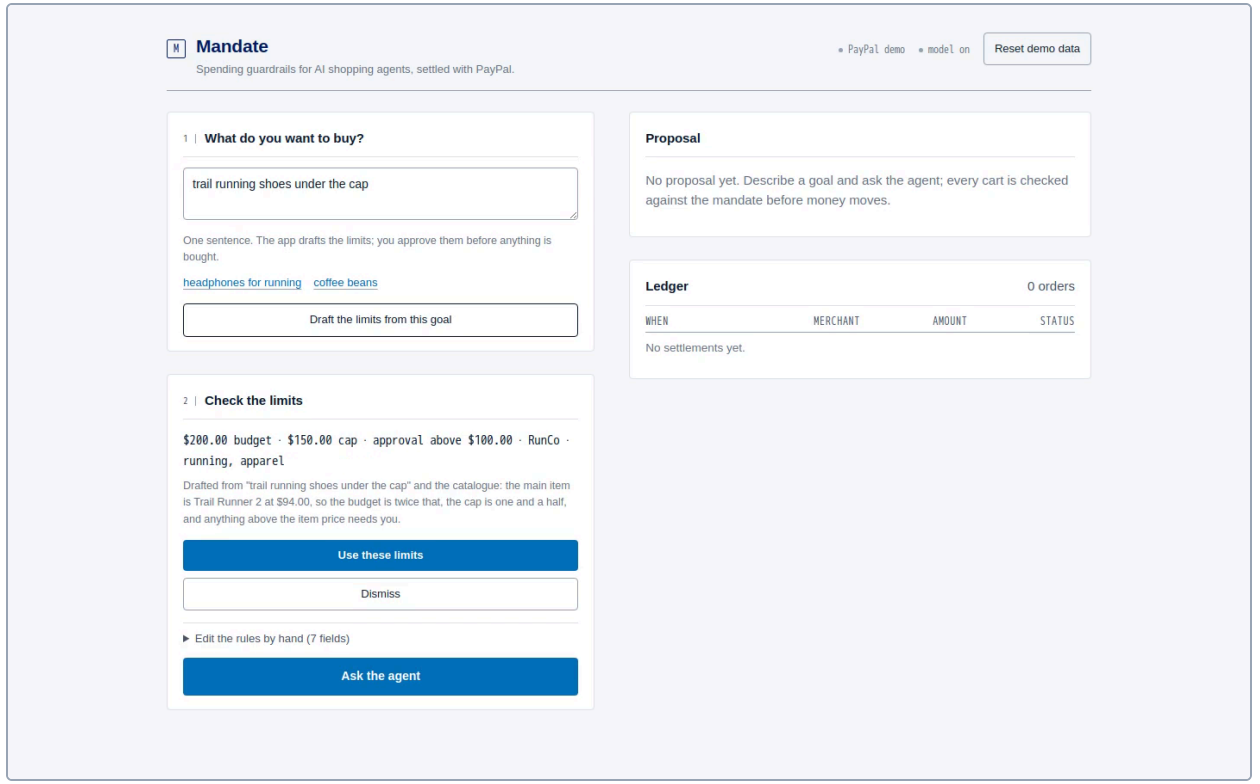
Every cart is checked; nothing moves without the rules.

The demo shows the same flow at desktop size. The form is behind a disclosure because enforcement, not authoring, is the product.

One screen, three decisions, no wizard.

- **Say the goal.** One sentence; the app does the authoring.
- **Approve the limits.** Budget, cap, threshold, merchant, categories — drafted, then one tap.
- **See the decision.** The stamp, the total and the reason sit in one card; the ledger is the record.

The seven fields stay one click away for anyone who wants them, which is what makes the one-tap path honest.



PayPal is the money path, not a badge.

- **OAuth 2.0** client-credentials token, cached until expiry.
- **Orders v2** create with `intent: CAPTURE` and integer-cent amounts.
- **Capture** happens after the buyer approves; the app captures on return.
- The server re-evaluates policy at checkout, so calling the endpoint directly cannot skip a rule.

Sandbox is the environment. Demo mode is a branch at the PayPal boundary, not a mock of the app.

OBSERVED IN SANDBOX	VALUE
Order created	57874901BY376944J
Capture id	5KT45525KN917902M
Captured before approval	422 ORDER_NOT_APPROVED
Amount	115.00 USD

PayPal itself refused a premature capture. That refusal is the proof the integration is live.

The model chooses. It never decides.

- **What the model does:** picks the item for the goal and writes the rationale, from a candidate list already filtered by merchant, category and cap.
- **What it cannot do:** set a price, invent a SKU, or authorise anything.
- **Evidence:** every proposal carries `engine` — `llm` only when the model really answered, `heuristic` when the fallback took over.
- **Model:** deepseek-v4.1-flash through an OpenAI-compatible gateway.

GOVERNANCE IN CODE	WHERE
Candidates pre-filtered by the mandate	`agent.ts`
SKUs validated against the catalogue	`agent.ts`
Prices come from the catalogue only	`catalog.ts`
Decision made by a pure function	`policy.ts`

A model that invents a SKU falls back to the deterministic planner, and a test proves it.

One function decides. Nothing else does.

`evaluate(proposal, mandate, now)` is pure: no I/O, no network, no wall-clock read on the decision. It returns an outcome, totals, the remaining budget, and reasons with codes and severities.

Checkout reloads the proposal and the mandate and runs it again, so a stale or forged decision is impossible. A denied cart answers `409 denied_by_policy` even when the caller sends `approved: true`.

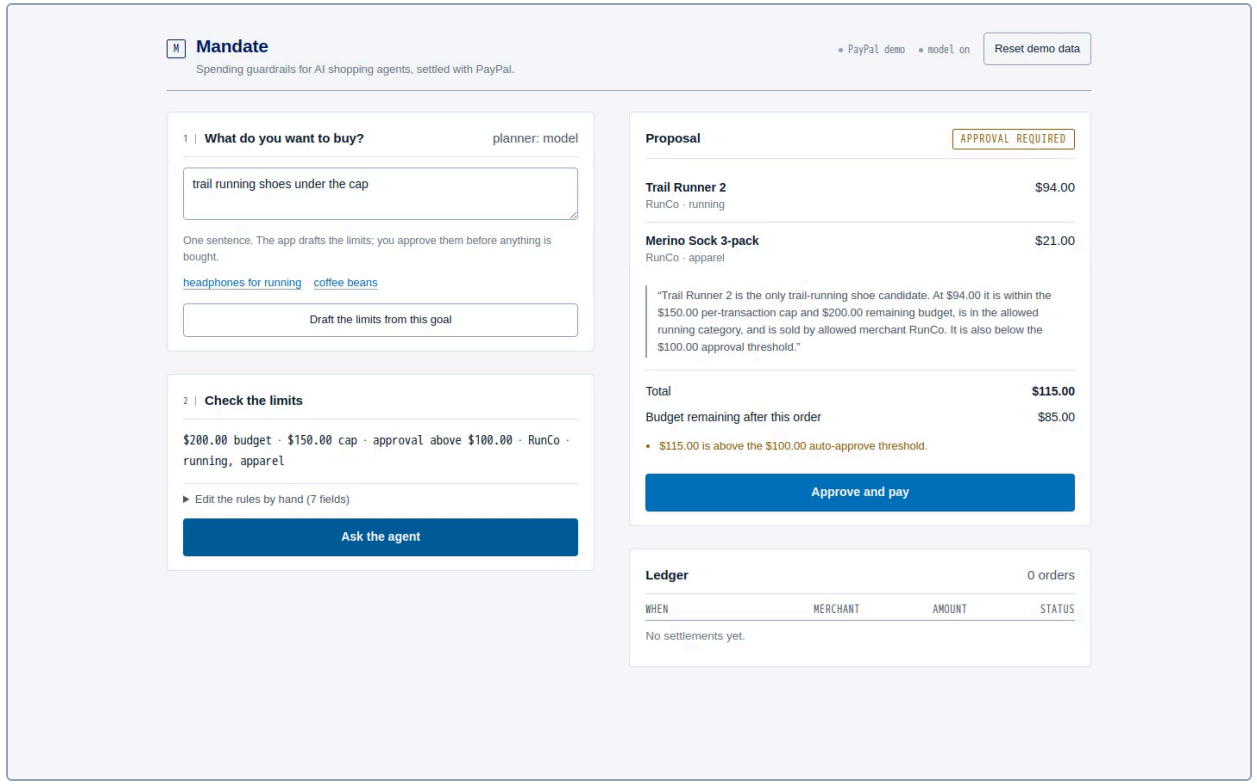
DENY · FOUR CODES AT ONCE

over_budget	`block`
over_transaction_cap	`block`
merchant_blocked	`block`
category_not_allowed	`block`

Audit needs a code, not a paragraph: `over_budget` is greppable and storable with the order.

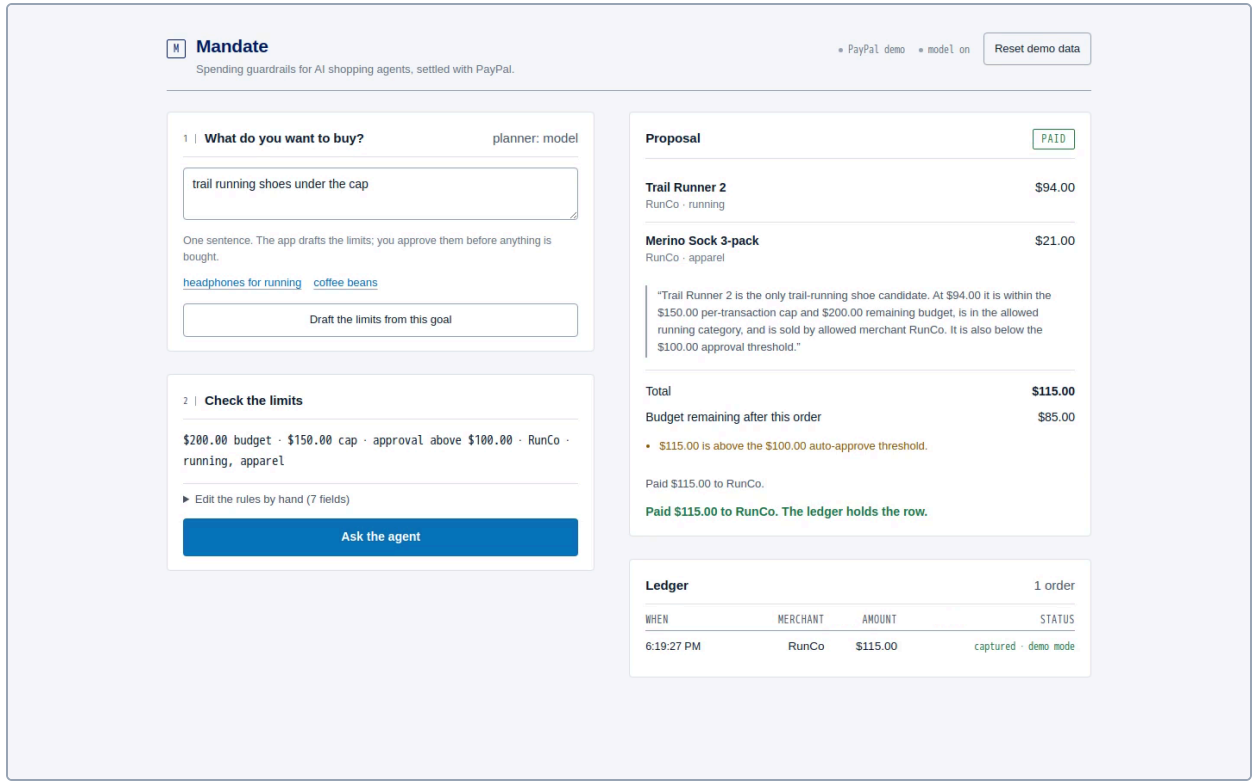
The cart that lands on the line.

- Two items, one merchant, **$115.00**.
- Under the $150 cap, inside the $200 budget, and above the $100 approval line.
- The stamp says **approval required**; the reason is a sentence a person can read.
- Nothing has moved. No order exists until a human acts.



Approved, settled, and written down.

- The stamp becomes **PAID**; the amount is stated in the same slot.
- One ledger row: merchant, amount, `captured`.
- The budget decrements: **$115.00 of $200.00**, so the next cart meets the rule that remains.
- In sandbox the same path produces a real capture id.



What is checked, not claimed.

SUITE	TESTS
Policy engine (cent boundaries, multi-reason denies)	16
HTTP API (approval gate, policy bypass)	15
Agent (model path, fallbacks, invented SKUs)	12
Catalogue search	6
PayPal client	5
Total	**54**

- CI green on the head commit: typecheck, tests, build.
- Sandbox capture recorded with a real capture id.
- No credentials needed: the whole demo runs in demo mode and says so on every row.
- One container builds, runs and serves the app and the pages.

Not another shopping agent.
The authority layer it was missing.

Most agentic-commerce demos show an agent buying something. Mandate shows the moment before: a deterministic engine ruling on a cart, with reasons a person can read and a machine can store.

The novel part is not the model. It is that the model has no vote, and that the refusal survives being repeated, replayed or called directly.

SAME GOAL, SECOND CART

```
Cart is $94.00 but only $85.00 remains.
```

A virtual card with a $200 limit cannot say that. A per-cart policy can, and it can say it again for the next cart.

Run it in thirty seconds.

```
git clone https://github.com/lgf2111/mandate
npm install && npm run dev
open http://localhost:5173
```

No credentials. Demo orders are simulated and labelled. With a PayPal sandbox app and an LLM key in `.env`, the same build creates real Orders v2 orders and asks a real model.

1
pure function decides

2
platforms: PayPal Orders v2 + the web app

0
rules the model can rewrite